HOSPITAL MANAGEMENT SYSTEM

A Python CLI System for Hospital Patient, Doctor, Appointment, and Billing Management

Student Name: Arukoti Bhavishya

Roll Number: 160625748005

College: Stanley College of Engineering and Technology for Women

Course: B.E CSE - AIML

GitHub Repository:

https://github.com/Bhaviicore04/hospital-management-system

1. PROJECT OVERVIEW

- Manage core hospital operations — patients, doctors, appointments, and billing — from a single command-line application
- Demonstrate understanding of Python fundamentals: file handling, dictionaries, lists, loops, and functions
- Use CSV files for persistent data storage without requiring a database
- Show modular program design with each feature (patient, doctor, appointment, billing) handled by its own set of functions
- Provide a live dashboard summarizing hospital activity in real time

2. HOW THE SYSTEM WORKS

1. User starts the application; default doctors are seeded automatically on first run
2. Dashboard displays a live summary: total patients, admitted/discharged counts, doctors on staff, scheduled appointments, and revenue
3. User selects a module from the Main Menu: Patient Management, Doctor Management, Appointment Management, or Billing
4. Within Patient Management, the user can register, view, search, update, or discharge a patient
5. Within Doctor Management, the user can view, add, or remove doctors
6. Within Appointment Management, the user can book, view, search, or cancel appointments
7. Within Billing, the user can generate an itemized bill for a patient and view total revenue across all bills

8. All changes are written immediately to the corresponding CSV file so data persists across sessions

3. TEST CASES

Test Case 1: Register New Patient

Input	Name: Aditi Rao, Age: 29, Gender: F, Contact: 9876543210, Address: Hyderabad, Blood Group: O+
Expected Output	✓ Patient registered successfully! Patient ID generated (e.g. P4821)
Why	All required fields were provided, so the record is written to patients.csv with status 'Admitted'

Test Case 2: Search Patient — Not Found

Input	Enter Patient ID or Name: 'XYZ999'
Expected Output	✗ No matching patient found.
Why	No record in patients.csv has a matching ID or name substring

Test Case 3: Discharge Already-Discharged Patient

Input	Enter Patient ID of a patient whose status is already 'Discharged'
Expected Output	⚠ Patient is already discharged.
Why	The system checks current status before allowing a second discharge

Test Case 4: Book Appointment with Valid Patient and Doctor

Input	Patient ID: P4821, Doctor ID: D1002, Date: 2026-06-25, Time: 10:00 AM, Reason: Fever
Expected Output	✓ Appointment booked! Appointment ID generated (e.g. A2390), status: 'Scheduled'
Why	Both the patient and doctor IDs exist, so the appointment is appended to appointments.csv

Test Case 5: Generate Bill with Invalid Amount

Input	Consultation Fee: 'abc' (non-numeric)
Expected Output	✗ Invalid amount entered.
Why	The system expects a numeric value and raises a ValueError on non-numeric input, which is caught and reported

Test Case 6: Exit Application

Input	User selects option 0 from the Main Menu
Expected Output	Screen clears; 'Thank you for using CityCare HMS. Goodbye!' is shown; program terminates
Why	Option 0 breaks the main loop, ending the program cleanly

4. LEARNING OUTCOMES

- File Handling: Reading and writing structured records using Python's csv module
- Data Structures: Organizing records with dictionaries and lists for fast lookup and update
- String Formatting: Building clean, aligned command-line tables and receipts
- Control Flow: Using loops and conditionals to drive menu navigation and validation
- Function Design: Splitting the system into single-responsibility functions per module
- State Management: Tracking patient and appointment status changes across sessions
- Problem Solving: Handling edge cases such as missing records and invalid numeric input

5. SCREENSHOTS

Insert screenshots of the following screens here before submitting:

- Screenshot 1: Dashboard / Main Menu
- Screenshot 2: Patient Registration
- Screenshot 3: Appointment Booking
- Screenshot 4: Bill Generation Receipt
- Screenshot 5: Exit Screen

OUTPUT

The Hospital Management System is a command-line application built in Python that allows hospital staff to register and manage patients, maintain a doctor directory, schedule and track appointments, and generate itemized bills with automatic revenue tracking — all backed by simple, persistent CSV file storage.